

DAY 3 - AI WORKSHOP

Machine Learning Fundamentals

Introduction to Machine Learning | Supervised & Unsupervised Learning

Session Agenda

Agenda

1

What is Machine Learning?

2

Types of ML & ML Workflow

3

Scikit-learn & Linear Regression

4

Classification Algorithms

5

Clustering with K-Means

6

Train-Test Split & Feature Selection

7

Hands-on: Scikit-learn Practice

8

Recap

Introduction to Machine Learning

What is Machine Learning?

"Machine Learning is a subset of AI that enables systems to learn and improve from experience without being explicitly programmed."

Traditional Programming

Rules + Data $\rightarrow$ Output

Machine Learning

Data + Output $\rightarrow$ Rules

Deep Learning

Large data + Neural Nets

AI $\supset$ Machine Learning $\supset$ Deep Learning

Real-World Applications of ML



Recommendation

Netflix, YouTube, Spotify suggestions



Healthcare

Disease diagnosis, drug discovery



Self-Driving

Tesla, Waymo autonomous vehicles



NLP / Chatbots

Siri, Alexa, ChatGPT, Claude



Finance

Fraud detection, credit scoring



Computer Vision

Face recognition, object detection

Types of Machine Learning



Supervised Learning

Learns from labeled data

- Linear Regression
- Logistic Regression
- Decision Trees
- SVM, KNN



Unsupervised Learning

Finds patterns in unlabeled data

- K-Means Clustering
- PCA
- Autoencoders
- DBSCAN

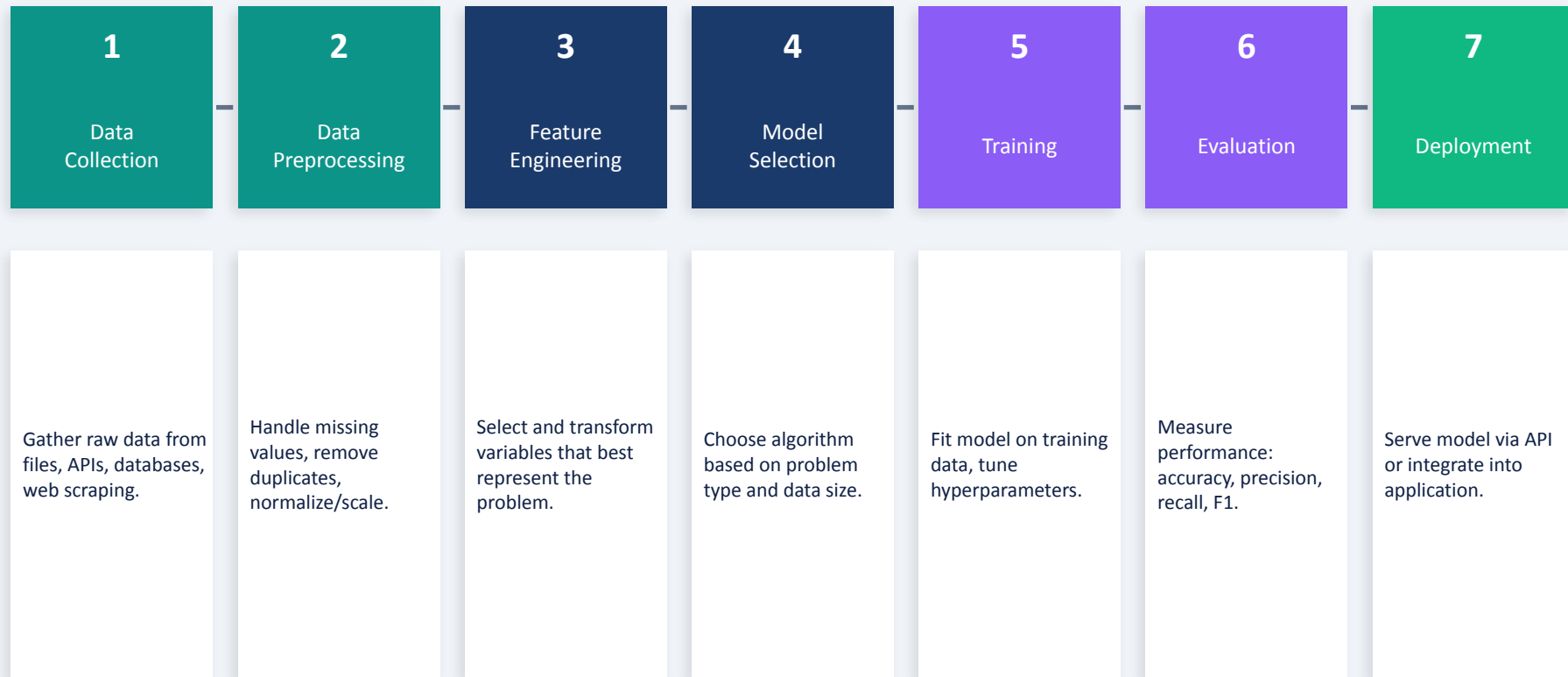


Reinforcement Learning

Learns via rewards & penalties

- Game AI (AlphaGo)
- Robotics
- Self-driving cars
- Trading bots

The Machine Learning Workflow



Introduction to Scikit-learn

Why Scikit-learn?

- Simple & consistent API
- Built on NumPy & SciPy
- 100+ ML algorithms
- Great documentation
- Industry standard tool

Typical Scikit-learn Template

```
from sklearn.linear_model import  
    LinearRegression  
  
# Create model  
model = LinearRegression()  
  
# Train  
model.fit(X_train, y_train)  
  
# Predict  
predictions = model.predict(X_test)  
  
# Score  
model.score(X_test, y_test)
```

Simple Linear Regression (Hands-on)

$$y = mx + b \rightarrow y = \beta_0 + \beta_1 x$$

Key Concepts

- $y \rightarrow$ Dependent variable (target)
- $x \rightarrow$ Independent variable (feature)
- $\beta_0 \rightarrow$ Intercept (bias)
- $\beta_1 \rightarrow$ Slope (coefficient/weight)
- Goal: Minimize prediction error (MSE)
- Used for: Price, Sales, Salary prediction

Hands-on Code

```
import numpy as np
from sklearn.linear_model import
    LinearRegression

# Sample data
X = np.array([[1],[2],[3],[4]])
y = np.array([2, 4, 5, 8])

model = LinearRegression()
model.fit(X, y)
print(model.coef_, model.intercept_)
```

Supervised and Unsupervised Learning

Classification Algorithms

Logistic Regression

- Classifies data into categories (0 or 1)
- Uses Sigmoid function: $\sigma(z) = 1/(1+e^{-z})$
- Output: Probability between 0 and 1
- Binary: Spam/Not-Spam, Disease/Healthy
- Multi-class: Digit recognition (0-9)
- Simple, fast, interpretable baseline

K-Nearest Neighbors (KNN)

- Non-parametric, instance-based method
- Classifies based on K nearest points
- Distance metric: Euclidean, Manhattan
- Small K $\rightarrow$ complex boundary (overfit)
- Large K $\rightarrow$ smooth boundary (underfit)
- Best K: use cross-validation
- Use case: Image recognition, recommendation

Classification - Code Example

Logistic Regression & KNN with Scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Logistic Regression
lr = LogisticRegression(); lr.fit(X_train, y_train)
print(f"LR Accuracy: {lr.score(X_test, y_test):.2f}")

# KNN
knn = KNeighborsClassifier(n_neighbors=5); knn.fit(X_train, y_train)
print(f"KNN Accuracy: {knn.score(X_test, y_test):.2f}")
```

Clustering with K-Means

K-Means Algorithm

1. Choose K (number of clusters)
2. Randomly place K centroids
3. Assign each point to nearest centroid
4. Recalculate centroid positions
5. Repeat until centroids stop moving

-  **Unsupervised:** no labels needed
-  **Choosing K:** Use Elbow Method
-  **Use cases:** Customer segmentation, document clustering, image compression.

K-Means Code

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Fit K-Means with k=3
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)

labels = kmeans.labels_
centers = kmeans.cluster_centers_

# Elbow method
inertia = []
for k in range(1,11):
    km = KMeans(k), km.fit(X)
    inertia.append(km.inertia_)
```

Train-Test Split

Why Split? → To evaluate how well the model generalizes to unseen data (avoid overfitting)

TRAINING DATA (80%)

TEST
(20%)

Training Set

Model learns patterns from this data.
Typically 70-80% of total data.



More data = better learning

Test Set

Used to evaluate final model
performance. Never seen during
training.



Simulates real-world data

Validation Set

Optional. Used to tune
hyperparameters during training phase.



Prevents overfitting during tuning

Feature Selection Basics

Feature Selection = Choosing the most relevant input variables to improve model accuracy and reduce complexity

Filter Methods

- Statistical test between feature & target.
- Correlation coefficient.
- Chi-square test.
- Fast & model-independent.

Wrapper Methods

- Use model to evaluate feature subsets.
- Forward/Backward selection.
- Recursive Feature Elimination (RFE)
- Computationally expensive.
- Example: sklearn RFE with SVM

Embedded Methods

- Selection happens during training
- LASSO (L1 regularization)
- Ridge (L2 regularization)
- Decision Tree feature importance
- Best of both worlds: speed & accuracy.

Hands-on Practice Session

Task 1

Load & Explore the Iris Dataset

- Load using `sklearn.datasets.load_iris()`
- Print shape, feature names, target names
- Explore with pandas: `df.head()`, `df.describe()`

Task 2

Train a Classifier (LR or KNN)

- Split data 80/20 with `train_test_split`
- Train Logistic Regression or `KNeighborsClassifier`
- Print accuracy score on test set

Task 3

K-Means Clustering

- Apply `K-Means(n_clusters=3)` on iris features
- Compare cluster labels vs true labels
- Try Elbow method to find best K

Bonus

Feature Selection

- Use correlation matrix to find top features
- Retrain model with only 2 best features
- Compare accuracy with/without selection



What is Machine Learning?

AI $\supset$ ML $\supset$ DL · Learns from data · Real-world applications in every industry.



Types & Workflow

Supervised · Unsupervised · Reinforcement · 7-step ML pipeline



Scikit-learn & Linear Regression

`.fit()` $\rightarrow$ `.predict()` $\rightarrow$ `.score()` · $y = \beta_0 + \beta_1 x$ · Minimize MSE



Classification

Logistic Regression: Sigmoid function · KNN: K nearest neighbors · Accuracy metric.



Clustering (K-Means)

Unsupervised grouping · Centroid iteration · Elbow method for K selection.



Train-Test Split & Feature Selection

80/20 split · Avoid overfitting · Filter · Wrapper · Embedded methods.

Thank You!

Un-Learn & Keep Learning. Keep Building.